

10 MVC deel 2 - Create, Update en Delete

📅 09-09-2026 ⌚ Ongeveer 13 minuten 🗨️ Ongeveer 1910 woorden

In het vorig hoofdstuk kwam volgende leerstof aan bod:

- Wat is het MVC-principe en hoe maak je een nieuw MVC-project.
- Hoe bouw je een View op aan de hand van HTML en Razor-code.
- Hoe haal je informatie uit een database en toon je de informatie op een View. In het eerste voorbeeld werd een volledig klant-model doorgegeven aan de View. In dit hoofdstuk leer je ViewModels gebruiken om gegevens door te geven tussen Views en Controllers. Er worden verschillende situaties aangereikt waar ViewModels gebruikt worden.

Setup

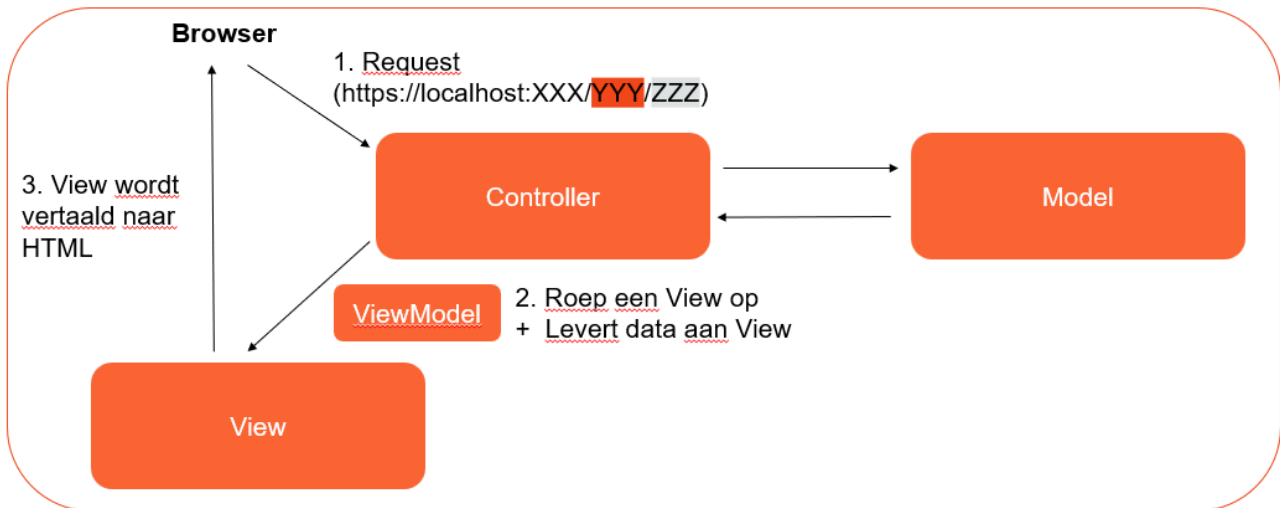
Clone het project MVCDemo.

ViewModels

Het doorgeven van een volledig model heeft dezelfde nadelen als bij een WebAPI:

- Je geeft overbodige informatie door wat je prestaties negatief beïnvloedt.
- Een volledig model kan gevoelige informatie bevatten.
- Je kunt problemen krijgen met validatie.
- Als het model verandert, moeten de Views ook worden aangepast.
- Moeilijker te testen.

Om die redenen heb je in H5 geleerd om DTO's te gebruiken. In MVC worden geen DTO's gebruikt, maar **ViewModels**. Het principe is ongeveer hetzelfde: In je ViewModel komen de Properties, data-annotations en methodes die je nodig hebt op je View. De Controller gebruikt het ViewModel om gegevens door te geven aan de View of om gegevens van een View (bv een formulier) te verwerken.



ViewModels plaats je steeds in een map `ViewModels`.

Formulier met zoekfunctie

In de vorige les werd uitgelegd hoe je een View maakt om een lijst te tonen van alle klanten. Stel dat je in deze lijst wilt filteren zodat je kunt zoeken op een deel van de voornaam of naam. De methode in de Controller werd al uitgelegd in H2 en herwerkt met `UnitOfWork` in H4. Om deze functionaliteit toe te voegen, volg je deze stappen:

1. Maak een nieuw ViewModel `KlantListViewModel.cs` in de map `ViewModels`. Je voorziet een string voor de zoekwaarde en een lijst van klanten.

```

1 namespace MVCDemo.ViewModels
2 {
3     public class KlantListViewModel
4     {
5         public string KlantSearch { get; set; }
6         public List<Klant> Klanten { get; set; }
7     }
8 }

```

CS

2. Pas de `KlantController` aan. In de `Index`-methode ga je de klanten ophalen en meegeven in het ViewModel.

```

1  public async Task<ActionResult<IEnumerable<Klant>>> Index()
2  {
3      //Klanten ophalen
4      var klanten = await _context.KlantRepository.GetAllAsync();
5      //ViewModel opvullen
6      KlantListViewModel model = new KlantListViewModel()
7      {
8          Klanten = klanten.ToList()
9      };
10     //ViewModel meegeven met de View
11     return View(model);
12 }

```

3. In de View pas je de verwijzing aan naar `KlantListViewModel` (regel 1) en ontwerp je het formulier (regels 7-16). Je gebruikt tag-helpers om het tekstvak te koppelen aan `KlantSearch`.

```

1  @model KlantListViewModel
2  @{
3      ViewData["Title"] = "Index";
4  }
5
6  <h2>Klanten</h2>
7  <form asp-controller="Klant" asp-action="Search" method="get">
8      <div class="form-actions no-color">
9          <p>
10             Zoek op klant:
11             <input asp-for="KlantSearch">
12             <input type="submit" value="Zoeken" class="btn btn-primary"
13         />
14             <a asp-controller="Klant" class="btn btn-primary" asp-
15             action="Index">Terug naar volledige lijst</a>
16         </p>
17     </div>
18 </form>
19 //Overige code

```

Je ziet dat er in het formulier een andere actie wordt uitgevoerd als je op *submit* klikt. Deze Action Method moet je nog aanmaken in de `KlantController` :

```
1 public async Task<IActionResult> Search(KlantListViewModel viewModel)    cs
2 {
3     if (!string.IsNullOrEmpty(viewModel.KlantSearch))
4     {
5         var klanten = await _context.KlantRepository.Find(x =>
6         x.Naam.Contains(viewModel.KlantSearch) ||
7         x.Voornaam.Contains(viewModel.KlantSearch) );
8         viewModel.Klanten = klanten.ToList();
9     }
10    else
11    {
12        var klanten = await _context.KlantRepository.GetAllAsync();
13        viewModel.Klanten = klanten.ToList();
14    }
15    return View("Index", viewModel);
16 }
```

Je krijgt als parameter het `KlantListViewModel` terug. Er wordt gecontroleerd of de zoekwaarde leeg is of niet. Indien de zoekwaarde leeg is, ga je in het ViewModel de volledige lijst van klanten opvullen. Als de zoekwaarde wel ingevuld is, vul je de lijst van klanten op met een gefilterde lijst.

Details opvragen en automapper

In het volgende voorbeeld ga je een ViewModel gebruiken om de details van een klant te tonen. Als de properties van je ViewModel overeenkomen met de properties uit één of meerdere **gerelateerde** modellen, is het interessant om de Automapper gebruiken. In het voorbeeld hierboven zat een zoekwaarde die nergens in een model voorkomt. In dat geval is het interessanter om je ViewModel handmatig op te vullen.

Onderstaand stappenplan wordt gebruikt voor de overige voorbeelden in dit hoofdstuk:

1. Maak een ViewModel aan.
2. Maak een mapping aan in `MapperProfile.cs`.
3. Breid je Controller uit.
4. Maak of wijzig je View.

1. Maak een nieuw ViewModel `KlantDetailsViewModel`. Je voorziet 2 properties: een samengestelde naam en de aangemaakte datum.

```

1 public class KlantDetailsViewModel
2 {
3     [Required]
4     public string Naam { get; set; }
5
6     [DataType(DataType.Date), Display(Name = "Datum Aangemaakt")]
7     public DateTime AangemaaktDatum { get; set; }
8 }

```

2. Maak de mapping aan in `MapperProfile.cs`. Hier ga je de naam samenstellen uit de voornaam en de achternaam.

```

1 CreateMap<Klant, KlantDetailsViewModel>()
2     .ForMember(dest => dest.Naam, opt => opt.MapFrom(src => $"
    {src.Voornaam} {src.Naam}"));

```

3. Breid de `KlantController` uit met een Details-actie. Je haalt eerst de klant op met `UnitOfWork`. Als je een klant gevonden hebt, map je de gegevens naar het `KlantDetailsViewModel` en dat geef je mee met je View. Als er geen klant gevonden is, ga je terug naar de Index-Actie.

```

1 public async Task<IActionResult> Details(int id)
2 {
3     Klant? klant = await _context.KlantRepository.GetByIdAsync(id);
4     if (klant != null)
5     {
6         KlantDetailsViewModel vm = _mapper.Map<KlantDetailsViewModel>
7         (klant);
8         return View(vm);
9     }
10    else
11    {
12        return RedirectToAction("Index");
13    }
14 }

```

4. Maak in de map `Views\Klant\` een nieuwe View met dezelfde naam als de actie: `Details`. Je gebruikt als model het `KlantDetailsViewModel` en daarna kun je de properties aanspreken in je code.

```
1 @model KlantDetailsViewModel
2 @{
3     ViewData["Title"] = "Details";
4 }
5
6 <h2> @Model.Naam werd aangemaakt op @Model.AangemaaktDatum</h2>
7
8 <a asp-controller="Klant" asp-action="Index" class="btn btn-
  primary">Terug</a>
```

Formulier met POST-request

In het eerste voorbeeld werd een GET-request gebruikt om het formulier door te geven aan de Controller. Het nadeel van een GET-request is dat de gegevens zichtbaar zijn in de url en dus kwetsbaar zijn. Deze methode is ook niet praktisch met een groot aantal properties. Het is daarom beter om je formulier terug te geven aan de Controller met een POST-request.

Als je werkt met formulieren heb je telkens 2 acties nodig in je Controller:

1. De eerste actie wordt gebruikt om het formulier te tonen op het scherm.
2. De tweede actie wordt gebruikt om het ingevulde formulier te verwerken in de Controller. Hier gebeurt vervolgens server-side validatie van het formulier.

In het volgende voorbeeld wordt het toevoegen van een klant uitgewerkt. Je werkt volgens het stappen plan:

1. Maak een nieuw ViewModel `KlantCreateViewModel` .

```
1 public class KlantCreateViewModel
2 {
3     [Required]
4     public string Naam { get; set; }
5     [Required]
6     public string Voornaam { get; set; }
7     [DataType(DataType.Date), Display(Name = "Datum Aangemaakt")]
8
9     public DateTime AangemaaktDatum { get; set; }
10 }
```

2. Maak de mapping aan in MapperProfile.cs .

```
1 CreateMap<KlantCreateViewModel, Klant>();
```

CS

3. Breid de `KlantController` uit met twee `Create`-acties maar bij de tweede actie plaats je het `[HttpPost]`-kenmerk om aan te geven dat deze `Create` de gegevens verwerkt van het formulier. De eerste actie laat alleen maar het formulier zien. In de tweede actie map je gegevens van het `KlantCreateViewModel` naar een object `Klant` en voeg je dat toe aan de database.

```
1 public IActionResult Create()
2 {
3     return View();
4 }
5
6 [HttpPost]
7 [ValidateAntiForgeryToken]
8 public async Task<IActionResult> Create(KlantCreateViewModel viewModel)
9 {
10     if (ModelState.IsValid)
11     {
12         Klant klant = _mapper.Map<Klant>(viewModel);
13         await _context.KlantRepository.AddAsync(klant);
14         _context.SaveChanges();
15         return RedirectToAction("Index");
16     }
17     return View(viewModel);
18 }
```

CS

4. Maak in de map `views\Klant\` een nieuwe View met dezelfde naam als de actie: `Create` . Je gebruikt als model het `KlantCreateViewModel` . Per property koppel je een label, een invoerveld en een validatieveld. Onderaan de pagina moet je een validatiescript in Javascript voorzien om client-side validatie te kunnen doen. Dit baseert zich op de data-annotations van je ViewModel. De foutboodschap komt dan in het validatieveld van de property.

```
1 @model KlantCreateViewModel
2 @{
3     ViewData["Title"] = "Create";
4 }
5
```

CS

```
6      <h1>Nieuwe klant aanmaken</h1>
7      <hr />
8      <div class="row">
9          <div class="col-md-4">
10             <form asp-action="Create">
11                 <div asp-validation-summary="ModelOnly" class="text-danger">
12                 </div>
13                 <div class="form-group">
14                     <label asp-for="Naam" class="control-label"></label>
15                     <input asp-for="Naam" class="form-control"/>
16                     <span asp-validation-for="Naam" class="text-danger">
17                 </span>
18                 </div>
19                 <div class="form-group">
20                     <label asp-for="Voornaam" class="control-label"></label>
21                     <input asp-for="Voornaam" class="form-control" />
22                     <span asp-validation-for="Voornaam" class="text-danger">
23                 </span>
24                 </div>
25                 <div class="form-group">
26                     <label asp-for="AangemaaktDatum" class="control-label">
27                 </label>
28                     <input asp-for="AangemaaktDatum" class="form-control" />
29                     <span asp-validation-for="AangemaaktDatum" class="text-
30 danger"></span>
31                 </div>
32                 <div class="form-group">
33                     <input type="submit" value="Create" class="btn btn-
34 primary" />
35                 </div>
36             </form>
37         </div>
38     </div>
    <div>
        <a asp-action="Index" class="btn btn-primary">Back to List</a>
    </div>
    @section Scripts{
        @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
    }
```

Deze werkwijze is gelijkaardig voor Edit- of Delete-acties: Je hebt telkens **twee** acties waarbij je eerst gegevens uit de database ophaalt en deze doorgeeft aan de View met behulp van een ViewModel. In de View toon je de gegevens in een formulier en daarna verwerk je de gegevens in de tweede actie van de Controller. Beide acties hebben dezelfde naam en de tweede actie krijgt als kenmerk **[HttpPost]**. Belangrijk ook om te melden is dat **alleen de properties van het ViewModel die ingevuld zijn op het formulier worden teruggegeven** naar de Controller. Om te weten welk record er moet gewijzigd worden of verwijderd, moet je de id van de entiteit in een **hidden-element** meegeven!

Uitgewerkte voorbeelden hiervan vind je in het MVCDemo-project en worden besproken tijdens de les.

ViewModels en relaties

Hieronder vind je een uitgewerkt voorbeeld om een lijst te tonen met alle bestellingen. Van elke bestelling wordt de id en de klantnaam getoond. Je haalt de gegevens uit de twee gerelateerde tabellen met een methode uit een specifieke BestellingRepository. Deze specifieke BestellingRepository is ook gedeclareerd in UnitOfWork. Je gebruikt dezelfde werkwijze als hierboven:

1. Maak een ViewModel aan. Voor deze situatie maak je twee ViewModellen aan. De eerste is voor een individueel record van een bestelling. De tweede is voor een lijst van bestellingen.

```
1      public class BestellingViewModel                                CS
2      {
3          public int Id { get; set; }
4          public string Naam { get; set; }
5      }
6
7      public class BestellingListViewModel
8      {
9          public List<BestellingViewModel> Bestellingen { get; set; }
10     }
```

2. Maak de mapping aan in MapperProfile.cs voor het BestellingViewModel .

```
1      CreateMap<Bestelling, BestellingViewModel>()                  CS
2          .ForMember(dest => dest.Naam, opt => opt.MapFrom(src => $"
```

```
{src.Klant.Voornaam} {src.Klant.Naam}")));
```

3. Maak een Index-actie in de `BestellingController`. Merk op dat er in het ViewModel een List van het `BestellingViewModel` gemapped wordt.

```
1 public async Task<ActionResult> Index()
2 {
3     var bestellingen = await
4     _context.BestellingRepository.GetAllBestellingenAsync();
5
6     BestellingListViewModel model = new BestellingListViewModel();
7
8     model.Bestellingen = _mapper.Map<List<BestellingViewModel>>
9     (bestellingen);
10
11     return View(model);
12 }
```

4. Maak in de map `Views\Bestelling\` een nieuwe View met dezelfde naam als de actie: `Index`.

```
1 @model BestellingListViewModel
2 @{
3     ViewData["Title"] = "Overzicht bestellingen";
4 }
5 <h2> Bestellingen</h2>
6
7 <a asp-action="AddBestelling" class="btn btn-primary">Nieuwe
8 bestelling</a>
9
10 <table class="table">
11     <tr>
12         <td>
13             @Html.DisplayNameFor(v => v.Bestellingen[0].Id)
14         </td>
15         <td>
16             @Html.DisplayNameFor(v => v.Bestellingen[0].Naam)
17         </td>
18     </tr>
19     @foreach (var item in Model.Bestellingen)
20     {
21         <tr>
```

```
22         <td>
23             @Html.DisplayFor(modelitem => item.Id)
24         </td>
25         <td>
26             @Html.DisplayFor(modelitem => item.Naam)
27         </td>
28
29         <td><a asp-action="EditBestelling" asp-route-id="@item.Id"
30 class="btn btn-primary">Edit</a></td>
31         <td><a asp-action="GetBestelling" asp-route-id="@item.Id"
32 class="btn btn-primary">Details</a></td>
33         <td><a asp-action="DeleteBestelling" asp-route-id="@item.Id"
class="btn btn-primary">Delete</a></td>
        </tr>
    }
</table>
```

Ook voor het tonen van de details van één bestelling gebruik je deze methode. Je gebruikt meerdere ViewModels om alle gegevens te verzamelen en te mappen. Bekijk dit voorbeeld in het MVCDemo-project.

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeveTM